13.5 会话间通信（Agent-to-Agent）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250,000 tokens，输出 ~18,000 tokens（本章合计）

前面的章节里，每个会话是一个独立的对话容器。但实际使用中，Agent 常常需要和其他会话中的 Agent 协作——比如主 Agent 将编码任务委托给一个子 Agent（Sub-agent），或者查询另一个会话的历史记录。OpenClaw 通过一组**会话工具**（`sessions_list`、`sessions_history`、`sessions_send`）实现了这种跨会话通信能力。

> **衍生解释**：Agent-to-Agent（A2A）通信是多 Agent 系统中的核心概念。不同于传统的函数调用，A2A 通信中每个 Agent 都是独立的"智能体"，有自己的对话上下文和行为模式。它们之间的交互更像是两个人之间的对话，而非简单的 RPC 调用。OpenClaw 的 A2A 通信建立在会话系统之上——每个 Agent 运行在自己的会话中，通过 Gateway 中转消息。

13.5.1 `sessions_list` / `sessions_history` / `sessions_send` 工具

这三个工具构成了 A2A 通信的基础设施，分别负责**发现**、**查看**和**发送**。

sessions_list：会话发现

`sessions_list` 让 Agent 能够查看当前存在哪些会话，包括自己生成的子 Agent 会话和其他 Agent 的会话。

```
// src/agents/tools/sessions-list-tool.ts
const SessionsListToolSchema = Type.Object({
  kinds: Type.Optional(Type.Array(Type.String())),   // 过滤类型：main/group/cron/hook/no
  limit: Type.Optional(Type.Number({ minimum: 1 })),
  activeMinutes: Type.Optional(Type.Number({ minimum: 1 })),  // 只返回最近 N 分钟活跃的
  messageLimit: Type.Optional(Type.Number({ minimum: 0 })),    // 附带最近 N 条消息
});

export function createSessionsListTool(opts?: {
  agentSessionKey?: string;
  sandboxed?: boolean;
}): AnyAgentTool {
  return {
    name: "sessions_list",
    description: "List sessions with optional filters and last messages.",
    execute: async (_toolCallId, args) => {
      const cfg = loadConfig();
      const a2aPolicy = createAgentToAgentPolicy(cfg);

      // 沙箱 Agent 只能看到自己生成的子 Agent 会话
      const restrictToSpawned = opts?.sandboxed === true
        && visibility === "spawned" && ...;

      const list = await callGateway({
        method: "sessions.list",
        params: { limit, activeMinutes, spawnedBy: restrictToSpawned ? requesterInterna
      });

      for (const entry of sessions) {
        // 跨 Agent 访问：检查 A2A 策略
        const crossAgent = entryAgentId !== requesterAgentId;
        if (crossAgent && !a2aPolicy.isAllowed(requesterAgentId, entryAgentId)) {
          continue;  // 未授权则跳过
        }

        // 按类型过滤
        const kind = classifySessionKind({ key, gatewayKind, alias, mainKey });
        if (allowedKinds && !allowedKinds.has(kind)) continue;

        // 可选：附带最近消息
        if (messageLimit > 0) {
          const history = await callGateway({
            method: "chat.history",
```

```
            params: { sessionKey: resolvedKey, limit: messageLimit },
          });
          row.messages = stripToolMessages(rawMessages);  // 过滤掉工具消息
        }

        rows.push(row);
      }
      return jsonResult({ count: rows.length, sessions: rows });
    },
  };
}
```

每个返回的会话条目包含：会话键、类型、通道、标签、最后更新时间、模型覆盖、Token 用量等元数据。 `messageLimit` 参数允许 Agent 在列表中直接预览每个会话的最近消息，避免额外调用 `sessions_history` 。

sessions_history：历史查看

`sessions_history` 获取指定会话的对话历史。由于对话历史可能非常大，该工具内置了多层截断保护：

```ts
// src/agents/tools/sessions-history-tool.ts
const SESSIONS_HISTORY_MAX_BYTES = 80 * 1024;        // 总响应上限：80KB
const SESSIONS_HISTORY_TEXT_MAX_CHARS = 4000;         // 单条消息文本上限：4000 字符

export function createSessionsHistoryTool(opts?: {
  agentSessionKey?: string;
  sandboxed?: boolean;
}): AnyAgentTool {
  return {
    name: "sessions_history",
    description: "Fetch message history for a session.",
    execute: async (_toolCallId, args) => {
      // 1. 跨 Agent 访问控制
      if (isCrossAgent && !a2aPolicy.isAllowed(requesterAgentId, targetAgentId)) {
        return jsonResult({ status: "forbidden", error: "Agent-to-agent history denied.
      }

      // 2. 获取原始历史
      const result = await callGateway({
        method: "chat.history",
        params: { sessionKey: resolvedKey, limit },
      });

      // 3. 清洗：截断过长文本、移除图片 base64 数据、删除 thinkingSignature
      const sanitizedMessages = selectedMessages.map(
        (msg) => sanitizeHistoryMessage(msg)
      );

      // 4. 总大小保护：超过 80KB 则从头部丢弃旧消息
      const cappedMessages = capArrayByJsonBytes(
        sanitizedMessages, SESSIONS_HISTORY_MAX_BYTES
      );

      return jsonResult({
        sessionKey: displayKey,
        messages: cappedMessages.items,
        truncated: /* ... */,
      });
    },
  };
}
```

清洗函数 `sanitizeHistoryMessage()` 会执行以下处理：

处理	说明
文本截断	超过 4000 字符的文本内容截断并添加 `…(truncated)…` 标记
图片移除	删除 base64 编码的图片数据，替换为 `{ omitted: true, bytes: N }`
签名删除	移除 `thinkingSignature` 字段（加密签名数据极大且无用）
元数据清理	删除 `details` 、 `usage` 、 `cost` 等大型嵌套字段

sessions_send：消息发送

`sessions_send` 是 A2A 通信的核心——它向目标会话发送一条消息，触发目标 Agent 处理，并可选地等待回复。

```
// src/agents/tools/sessions-send-tool.ts
const SessionsSendToolSchema = Type.Object({
  sessionKey: Type.Optional(Type.String()),   // 目标会话键
  label: Type.Optional(Type.String()),         // 或通过标签定位
  agentId: Type.Optional(Type.String()),       // 标签查找的 Agent 范围
  message: Type.String(),                      // 消息内容
  timeoutSeconds: Type.Optional(Type.Number({ minimum: 0 })),  // 等待超时
});
```

该工具支持两种目标定位方式：通过 `sessionKey` 精确定位，或通过 `label` （会话标签）模糊查找。两者不可同时使用。

消息发送有两种模式：

即发即忘（Fire-and-Forget）： `timeoutSeconds = 0`

```
// src/agents/tools/sessions-send-tool.ts（即发即忘模式）
if (timeoutSeconds === 0) {
  const response = await callGateway({
    method: "agent",
    params: sendParams,
    timeoutMs: 10_000,
  });
  startA2AFlow(undefined, runId);  // 启动后台 A2A 流程
  return jsonResult({ runId, status: "accepted", sessionKey: displayKey });
}
```

等待回复： `timeoutSeconds > 0` （默认 30 秒）

```
// src/agents/tools/sessions-send-tool.ts（等待回复模式）
// 1. 发送消息，触发目标 Agent
await callGateway({ method: "agent", params: sendParams });

// 2. 等待目标 Agent 完成
const wait = await callGateway({
  method: "agent.wait",
  params: { runId, timeoutMs },
});

// 3. 读取目标 Agent 的最后回复
const history = await callGateway({
  method: "chat.history",
  params: { sessionKey: resolvedKey, limit: 50 },
});
const reply = extractAssistantText(lastMessage);

// 4. 启动 A2A 后续流程（ping-pong + announce）
startA2AFlow(reply);

return jsonResult({ runId, status: "ok", reply, sessionKey: displayKey });
```

跨 Agent 访问控制

三个工具都遵循统一的 **A2A 策略**控制。默认情况下，跨 Agent 的访问是**禁止**的——必须在配置中显式开启：

```
// openclaw.json5
{
  "tools": {
    "agentToAgent": {
      "enabled": true,           // 开启跨 Agent 通信
      "allow": [
        { "from": "main", "to": "coding-assistant" },
        { "from": "coding-assistant", "to": "main" }
      ]
    }
  }
}
```

`createAgentToAgentPolicy()` 会解析这个配置并生成策略对象，每次跨 Agent 操作都会检查 `isAllowed(fromAgentId, toAgentId)` 是否返回 `true` 。

13.5.2 跨会话消息传递与回复乒乓（Reply Ping-Pong）

`sessions_send` 不仅仅是发一条消息——它还支持一个完整的**多轮对话流程**，称为回复乒乓（Reply Ping-Pong）。这个机制允许两个 Agent 在收到消息后自动进行多轮交互，然后将最终结果发布到目标通道。

完整 A2A 流程

以下是一个 `sessions_send` 触发的完整 A2A 交互流程：

```
Agent A (requester)                Gateway                Agent B (target)
      |                                |                          |
      |--- sessions_send(msg) ------>|                          |
      |                                |--- agent(msg) --------->|
      |                                |<-- round 1 reply -------|
      |                                |                          |
      |    [Ping-Pong Phase]           |                          |
      |<-- reply context + msg ------|                          |
      |--- round 2 reply ----------->|                          |
      |                                |--- reply context ------->|
      |                                |<-- round 3 reply --------|
      |                                |   ... (up to 5 turns)    |
      |                                |                          |
      |    [Announce Phase]            |                          |
      |                                |--- announce context ---->|
      |                                |<-- final message --------|
      |                                |--- send to channel ----->|
      |                                |                          |
```

A2A 流程实现

`src/agents/tools/sessions-send-tool.a2a.ts` 中的 `runSessionsSendA2AFlow()` 实现了完整的乒乓和公告流程：

```typescript
// src/agents/tools/sessions-send-tool.a2a.ts
export async function runSessionsSendA2AFlow(params: {
  targetSessionKey: string;
  message: string;
  maxPingPongTurns: number;           // 最大乒乓轮次（默认 5）
  requesterSessionKey?: string;
  roundOneReply?: string;             // 第一轮回复（如果已等待获得）
}) {
  let latestReply = params.roundOneReply;
  if (!latestReply) return;  // 没有回复则终止

  // === 乒乓阶段 ===
  if (params.maxPingPongTurns > 0 && params.requesterSessionKey) {
    let currentSessionKey = params.requesterSessionKey;
    let nextSessionKey = params.targetSessionKey;
    let incomingMessage = latestReply;

    for (let turn = 1; turn <= params.maxPingPongTurns; turn += 1) {
      const currentRole = currentSessionKey === params.requesterSessionKey
        ? "requester" : "target";

      // 构建回复上下文提示
      const replyPrompt = buildAgentToAgentReplyContext({
        requesterSessionKey: params.requesterSessionKey,
        targetSessionKey: params.displayKey,
        currentRole,
        turn,
        maxTurns: params.maxPingPongTurns,
      });

      // 在当前 Agent 的会话中运行一步
      const replyText = await runAgentStep({
        sessionKey: currentSessionKey,
        message: incomingMessage,
        extraSystemPrompt: replyPrompt,
      });

      // Agent 可以回复 "REPLY_SKIP" 来终止乒乓
      if (!replyText || isReplySkip(replyText)) break;

      latestReply = replyText;
      incomingMessage = replyText;
```

```
      // 交换角色：requester ↔ target
      const swap = currentSessionKey;
      currentSessionKey = nextSessionKey;
      nextSessionKey = swap;
    }
  }

  // === 公告阶段 ===
  const announceTarget = await resolveAnnounceTarget({
    sessionKey: params.targetSessionKey,
  });

  const announcePrompt = buildAgentToAgentAnnounceContext({
    originalMessage: params.message,
    roundOneReply: primaryReply,
    latestReply,
  });

  const announceReply = await runAgentStep({
    sessionKey: params.targetSessionKey,
    message: "Agent-to-agent announce step.",
    extraSystemPrompt: announcePrompt,
  });

  // 发布到目标通道（除非 Agent 回复 "ANNOUNCE_SKIP"）
  if (announceTarget && announceReply && !isAnnounceSkip(announceReply)) {
    await callGateway({
      method: "send",
      params: {
        to: announceTarget.to,
        message: announceReply.trim(),
        channel: announceTarget.channel,
      },
    });
  }
}
```

乒乓上下文注入

每一轮乒乓都会注入一段额外的系统提示，告知当前 Agent 它正在参与一个 A2A 对话：

```ts
// src/agents/tools/sessions-send-helpers.ts
export function buildAgentToAgentReplyContext(params: {
  requesterSessionKey?: string;
  targetSessionKey: string;
  currentRole: "requester" | "target";
  turn: number;
  maxTurns: number;
}) {
  return [
    "Agent-to-agent reply step:",
    `Current agent: ${currentLabel}.`,
    `Turn ${params.turn} of ${params.maxTurns}.`,
    `Agent 1 (requester) session: ${params.requesterSessionKey}.`,
    `Agent 2 (target) session: ${params.targetSessionKey}.`,
    `If you want to stop the ping-pong, reply exactly "REPLY_SKIP".`,
  ].filter(Boolean).join("\n");
}
```

这段提示让 Agent 知道：

1. 自己当前的角色（请求方还是目标方）
2. 当前是第几轮
3. 可以通过回复 `REPLY_SKIP` 主动终止对话

乒乓轮次的上限由配置项 `session.agentToAgent.maxPingPongTurns` 控制，默认为 5，硬上限也是 5：

```ts
// src/agents/tools/sessions-send-helpers.ts
const DEFAULT_PING_PONG_TURNS = 5;
const MAX_PING_PONG_TURNS = 5;

export function resolvePingPongTurns(cfg?: OpenClawConfig) {
  const raw = cfg?.session?.agentToAgent?.maxPingPongTurns;
  return Math.max(0, Math.min(MAX_PING_PONG_TURNS, Math.floor(raw ?? DEFAULT_PING_PONG_
}
```

公告阶段

乒乓结束后进入公告阶段。目标 Agent 会收到一段包含完整上下文的提示，要求它生成一条面向最终用户的消息：

```ts
// src/agents/tools/sessions-send-helpers.ts
export function buildAgentToAgentAnnounceContext(params: {
  originalMessage: string;
  roundOneReply?: string;
  latestReply?: string;
}) {
  return [
    "Agent-to-agent announce step:",
    `Original request: ${params.originalMessage}`,
    `Round 1 reply: ${params.roundOneReply ?? "(not available)"}`,
    `Latest reply: ${params.latestReply ?? "(not available)"}`,
    `If you want to remain silent, reply exactly "ANNOUNCE_SKIP".`,
    "Any other reply will be posted to the target channel.",
    "After this reply, the agent-to-agent conversation is over.",
  ].filter(Boolean).join("\n");
}
```

目标 Agent 可以选择：

- **回复正常消息**：该消息会被通过 `send` 方法发布到目标通道（如 Telegram 群组、Slack 频道）
- **回复 `ANNOUNCE_SKIP`**：保持沉默，不向用户发送任何消息

本节小结

OpenClaw 的会话间通信体系提供了完整的多 Agent 协作能力：

1. **三工具基础设施**：`sessions_list`（发现）、`sessions_history`（查看）、`sessions_send`（发送）构成了 A2A 通信的完整原语
2. **细粒度访问控制**：跨 Agent 通信默认禁止，需要通过 `tools.agentToAgent` 配置显式授权，沙箱 Agent 的可见范围进一步受限
3. **回复乒乓机制**：两个 Agent 可以自动进行最多 5 轮的交互式对话，任一方可通过 `REPLY_SKIP` 提前终止
4. **公告阶段**：乒乓结束后，目标 Agent 可以将协作结果发布到用户可见的通道，或通过 `ANNOUNCE_SKIP` 保持沉默
5. **安全保护**：80KB 响应上限、历史文本截断、沙箱会话隔离等多层防护确保了 A2A 通信不会导致资源滥用

这套机制使得 OpenClaw 不仅是一个单一的 AI 助手，更是一个可以容纳多个协作 Agent 的平台。后续章节中，我们会看到子 Agent（Sub-agent）系统如何利用这些原语实现任务委托和结果汇报。